

# Customer Churn Prediction Model Deployment

Final Capstone Project & Real-Time REST API Documentation

|             |                                              |
|-------------|----------------------------------------------|
| Repository: | github.com/Harprabh-Singh/customer-churn-api |
| Author:     | Harprabh Singh                               |
| Date:       | August 15, 2026                              |
| Status:     | Production Ready / Verified                  |

*This document provides a detailed overview of the system architecture, code structure, deployment instructions, and the ongoing maintenance strategy designed for the customer churn prediction pipeline.*

# 1. System Architecture

The system is built as a complete end-to-end customer churn prediction pipeline, integrating offline machine learning training, real-time REST API inference, and batch overnight scoring.

## Real-Time Inference API (Flask)

The web service is built using Flask and serves real-time prediction requests. It loads the pre-trained Random Forest classifier and Scikit-Learn preprocessing pipeline once at startup to ensure low-latency responses. It exposes a single POST endpoint at /predict which accepts a nested JSON structure representing the customer profile.

## Batch Scoring Script

For offline nightly scoring, a robust Python command-line utility (batch.py) processes input customer datasets. It executes HTTP requests concurrently or sequentially against the live API, collects inference metrics, outputs predicted labels, and compiles monitoring logs.

# 2. Repository Structure & Artifacts

The workspace and code structure are structured exactly as defined in the capstone submission specification:

```
customer-churn-api/
├──
├── app/
│   ├── __init__.py      # Package initialiser
│   ├── main.py          # Flask REST API implementation
│   ├── utils.py         # Serialised artifact loaders & predictors
│   ├── model.pkl        # Serialised Random Forest Model
│   └── transformer.pkl   # Serialised Preprocessing ColumnTransformer
├──
├── test_data/
│   ├── all_customers.csv # Nightly batch scoring test dataset
│   └── sample_input.json # Sample REST API payload (nested format)
├──
├── logs/
│   └── batch_log.txt     # Run summary logs generated by batch.py
├──
├── batch.py             # Nightly batch scoring pipeline script
├── train.py             # Model training & serialization script
├── requirements.txt      # Python package dependencies
├── README.md            # Documentation & Maintenance Plan
└── .gitignore           # Git exclusion rules
```

### 3. Maintenance & Monitoring Plan

A robust machine learning pipeline requires continuous oversight to counter degradation, data drift, and environment changes. Below is the established three-pillar maintenance plan:

#### Retraining Strategy

The model should be retrained **monthly** or whenever churn rate in production deviates more than **±5 percentage points** from the training baseline. To retrain: update `test_data/all_customers.csv` with fresh labelled data, run `python train.py`, and redeploy the updated `.pkl` files. Use a version-controlled filename convention (e.g., `model_v2_2026-09.pkl`) and keep the previous artefact for rollback. Automate retraining via a cron job or CI/CD pipeline trigger. Always evaluate on a held-out test split and compare ROC-AUC against the current production model before promoting a new version. Minimum acceptable ROC-AUC is **0.75**.

#### Drift Detection & Monitoring

Monitor **data drift** monthly by comparing feature distributions (mean, std, category frequencies) of incoming API payloads against the training distribution. Use Population Stability Index (PSI) — flag any feature with  $PSI > 0.2$  for investigation. Monitor **concept drift** by tracking prediction distributions and, where labels are available (e.g., after 90 days), computing actual vs predicted churn rates. Log all incoming payloads to a database or data warehouse to enable retrospective analysis. Alert the team via email or Slack if drift thresholds are breached.

#### Model & Artifact Versioning

All code, training scripts, and serialised artefacts are version-controlled in this Git repository. Use **semantic versioning** for model releases (e.g., `v1.0.0`). Tag each release in Git (`git tag v1.0.0`) and store corresponding `.pkl` artefacts with matching version suffixes in a dedicated `models/` directory or an object storage bucket (e.g., AWS S3). Maintain a `CHANGELOG.md` documenting changes per version. Each deployment should reference an explicit model version so that rollback is a single config change. Never overwrite production artefacts in place without first confirming the new model passes evaluation criteria.

## 4. Step-by-Step Execution Guide

Follow these steps to run the pipeline end-to-end locally:

### Step 1: Install Dependencies

Install the required libraries listed in requirements.txt:

```
pip install -r requirements.txt
```

### Step 2: Train the Model

Train the RandomForest model on your clean dataset:

```
python train.py
```

### Step 3: Launch the Real-Time API

Run the Flask application as a package module:

```
python -m app.main
```

### Step 4: Run Batch Scorer

While the API is running, launch the nightly scoring script in another terminal:

```
python batch.py --input test_data/all_customers.csv
```

## Model Performance Verification Output

```
[INFO] Batch scoring started
[INFO] Input   : test_data/all_customers.csv
[INFO] Output  : scored_customers.csv
[INFO] API URL: http://localhost:8000/predict
...
[INFO] BATCH SUMMARY
[INFO] Total requests      : 30
[INFO] Successful calls    : 30
[INFO] Failed predictions  : 0
[INFO] Churn = Yes         : 11 (36.7%)
[INFO] Avg churn prob      : 0.4102
```